

An In-Network Timing Framework for Concealing the Response-Time Fingerprint of a Separate-Acknowledgment DNP3 Device

Anonymous submission

I. Introduction

An attacker studies a target before acting on it. Fingerprinting is how a passive observer turns raw network traffic into knowledge about the systems that produce it, learning a device’s type, its role, its vendor, or its behavior without ever decoding the payload [1]–[3]. This knowledge is the reconnaissance stage of an intrusion: it tells the attacker which hosts matter and how they will respond. The concern is acute for industrial control systems and the power grid, where the devices are long-lived, their roles are stable, and prior work has fingerprinted supervisory and field equipment directly from its traffic [4], [5]. One such feature is the cross-layer response time, the delay between a request and the device’s reply, which Formby et al. showed distinguishes control-system devices by vendor and model [5]. Because a legacy protocol such as DNP3 carries this timing in the clear, an on-path observer can read the fingerprint from ordinary polling traffic.

Traffic obfuscation defends against such analysis by shaping what the observer sees. Existing work shapes packet size, inter-packet timing, sending rate, or the overall communication pattern [6]–[9], and stronger analyses have repeatedly defeated the weaker of these countermeasures [10], [11]. Many of these defenses were designed for the Internet setting, where the traffic is already encrypted or otherwise payload-opaque. There, the shaper is free to add cover packets or reshape flows without contradicting anything the observer can read [6], [8]. Legacy industrial traffic breaks that assumption. DNP3 exchanges are plaintext and each application block carries a validating checksum, so an observer can tell a genuine protocol exchange from an injected dummy or an altered response [12], [13]. A defense that transforms this traffic therefore cannot rely on cover payloads or byte edits, and the techniques built for encrypted flows do not transfer directly.

The result is a gap. The field devices are vendor-supplied and cannot be reprogrammed in practice, and the deployed protocol cannot be changed [12], [14], so a practical defense must sit in the network and remain transparent to both endpoints. Programmable switches make such in-network

shaping feasible at line rate [15]–[17], but a defense for plaintext DNP3 must satisfy constraints those general shapers do not. It must preserve the two endpoints, the DNP3 request and response exchange, the original packet bytes, and the correctness of every delivered reply. It must also keep any added delay inside the DNP3, TCP, polling, and quality-of-service limits so the control loop still functions. We assume the switch is a trusted observation and control point on the plaintext path, and that the adversary is a passive observer on the master-facing segment, where only the released, shaped timing is visible. Within these constraints, the response-time fingerprint remains open, because the size-oriented and encrypted-flow defenses above do not address the request-to-reply timing of a legacy device that cannot be modified.

We present an in-network framework that conceals the response-time fingerprint of a separate-acknowledgment DNP3 device without touching the device, the protocol, or the packet contents. A single programmable switch sits between the DNP3 master and the field device and observes the events of each polling transaction. Rather than editing any packet, it controls when the device’s original acknowledgment and response become externally visible. The original packets stay resident in the switch’s queues while small internal marker packets recirculate to hold their place. The traffic manager then releases the held packets on a schedule the control plane sets. This design works with, rather than against, the fact that a data-plane pipeline cannot recall an already-enqueued packet at an arbitrary later time chosen in software. Configurable gates on the acknowledgment and on the response give several transformations of the observed timing: an event-driven mode that releases the acknowledgment only when the matching response appears, an acknowledgment-deadline mode, and a dual-deadline mode that holds both. These are modes of one framework rather than separate systems, and the control plane selects among them and sets the delays at run time.

Contributions. This paper makes the following contributions, each bounded by the evidence we report.

- We frame the cross-layer response time, identified by Formby et al. [5] as a device-fingerprinting feature,

as a reachable in-network shaping target for legacy DNP3 devices, and we frame concealing it as an in-network shaping problem under the plaintext, byte-preserving constraints of the setting.

- We design a unified, queue-resident timing framework that controls the visibility of the original acknowledgment and response through traffic-manager scheduling, and we show how the event-driven, acknowledgment-deadline, and dual-deadline transformations arise as configurations of one mechanism. We also report where the response-only configuration does not shape a separate-acknowledgment device, and treat that as an explicit boundary.
- We resolve the DNP3, TCP, and Tofino constraints the design meets: matching each transaction to its DNP3 transaction-generation code on plaintext traffic, preserving the original bytes, keeping the internal markers off the observable wire, and fitting the arithmetic within a single Tofino-1 ingress pipeline.
- We implement the framework on an Intel Tofino-1 switch and evaluate it on silicon against a physical SEL-751 relay. The dual-deadline mode normalizes the measured response time from a variable native distribution to a fixed value, the acknowledgment and response order is preserved, no original packet is dropped or altered, and the runtime fail-open path releases held packets within a bounded time. We report these results, and their limits, for one relay, for READ polling transactions, and for the response-time observable only; we do not claim device indistinguishability, size concealment, or a general multi-flow defense.

References

- [1] T. Kohno, A. Broido, and K. C. Claffy, “Remote physical device fingerprinting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [13].
- [2] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, “GTID: A technique for physical device and device type fingerprinting,” *IEEE Transactions on Dependable and Secure Computing*, 2014, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [18].
- [3] G. Shu and D. Lee, “Network protocol system fingerprinting – a formal approach,” in *Proceedings IEEE INFOCOM 2006*, 2006, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [19].
- [4] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, “Passive fingerprinting of SCADA in critical infrastructure network without deep packet inspection,” 2016, verified via arXiv MCP (export_citations). [Online]. Available: <https://arxiv.org/abs/1608.07679>
- [5] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, “Who’s in control of your control system? device fingerprinting for cyber-physical systems,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016, anchor paper, read in full. The CLRT device-fingerprinting threat model.
- [6] C. V. Wright, S. E. Coull, and F. Monrose, “Traffic morphing: An efficient defense against statistical traffic analysis,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009, metadata verified from Ditto (NDSS 2022) bibliography, ref. [100].
- [7] X. Cai, R. Nithyanand, and R. Johnson, “CS-BuFLO: A congestion sensitive website fingerprinting defense,” in *Proceedings of the 13th Workshop on Privacy in the Electronic Society (WPES)*, 2014, venue corroborated by Ditto (NDSS 2022) ref. [36]; metadata via arXiv MCP (1401.6022, “New Approaches to Website Fingerprinting Defenses”).
- [8] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, “Toward an efficient website fingerprinting defense,” in *Computer Security – ESORICS 2016*, 2016, wTF-PAD. Venue corroborated by Ditto (NDSS 2022) ref. [67]; metadata via arXiv MCP (1512.00524).
- [9] N. Aphorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, “Keeping the smart home private with smart(er) IoT traffic shaping,” *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019, stochastic Traffic Padding (STP). Venue corroborated by Ditto (NDSS 2022) ref. [26]; metadata via arXiv MCP (1812.00955).
- [10] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, “Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail,” in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012, buFLO. Metadata verified from Ditto (NDSS 2022) bibliography, ref. [50].
- [11] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, venue corroborated by Ditto (NDSS 2022) ref. [92]; metadata via arXiv MCP (1801.02265).
- [12] S. East, J. Butts, M. Papa, and S. Sheno, “A taxonomy of attacks on the DNP3 protocol,” in *Critical Infrastructure Protection III (IFIP AICT, vol. 311)*. Springer, 2009, pp. 67–81, web-verified (Springer / IFIP CIP III).
- [13] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, “Modbus/DNP3 state-based intrusion detection system,” in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [10].
- [14] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, “Attacks against process control systems: Risk assessment, detection, and response,” in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [6].
- [15] L. Wang, H. Kim, P. Mittal, and J. Rexford, “Programmable in-network obfuscation of traffic,” 2020, verified via arXiv MCP (get_abstract). PINOT; Tofino prototype. [Online]. Available: <https://arxiv.org/abs/2006.00097>
- [16] R. Meier, V. Lenders, and L. Vanbever, “ditto: WAN traffic obfuscation at line rate,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022, anchor paper, read in full. Closest in-network line-rate obfuscation prior work.
- [17] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, “An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends,” 2021, verified via arXiv MCP (export_citations). Also in *IEEE Access*, 2021. [Online]. Available: <https://arxiv.org/abs/2102.00643>